

시나리오

PG 서비스 기준으로 “실시간 승인 모니터링 + 정산 배치”를 기존 RDBMS보다 빠르게 보여 주려면 POC 시나리오를 단순히 데이터레이크 구축 → 쿼리 흐름으로 하면 안 됩니다.

핵심은 **Streaming + Incremental Lakehouse + MPP Query** 구조를 활용해서 **RDBMS에서 어려운 부분을 강조**하는 겁니다.

지금 구성은 사실 **정석적인 Lakehouse 아키텍처**입니다.

- Storage: **MinIO (S3)**
- Table Format: **Iceberg**
- Catalog: **Lakekeeper**
- Streaming Compute: **Flink**
- Batch Compute: **Spark**
- Ingest: **Kafka**
- Query Engine: **Trino**

이걸 활용한 **PG 데이터 플랫폼 POC 시나리오**를 실제 구축 Step 기준으로 설명할게요.

1 POC 목표 정의 (중요)

POC 목표는 **RDBMS보다 잘하는 것만** 보여주는 것입니다.

PG 기준으로 RDBMS가 약한 영역

- 1 실시간 승인 모니터링
- 2 대용량 집계
- 3 기간별 정산
- 4 과거 데이터 분석

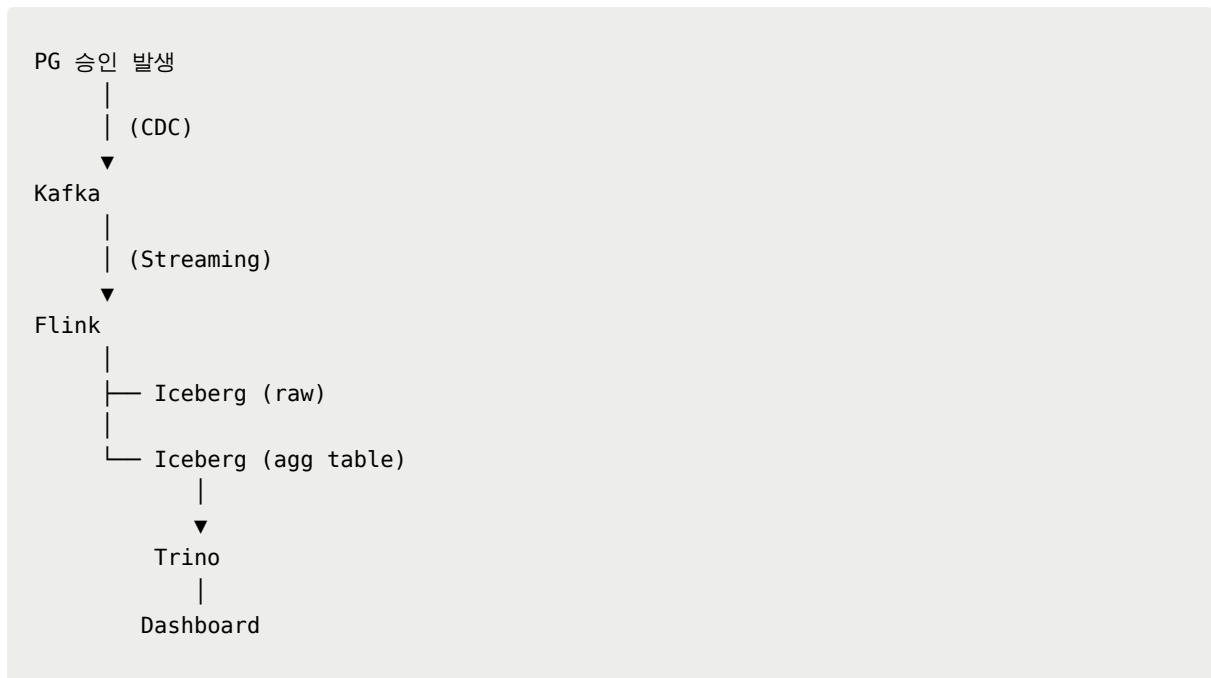
그래서 목표는 이렇게 잡는 게 좋습니다.

시나리오	목표
승인 실시간 모니터링	승인 발생 후 3초 내 Dashboard 반영
시간별 승인량 집계	수억건 데이터도 1초 조회
정산 배치	기존 배치 30분 → 1~3분

시나리오	목표
장기 데이터 분석	3년 데이터도 빠른 조회

2 전체 데이터 흐름 설계

POC에서 사용할 정석 구조



Spark는 정산 배치용으로 사용.

3 Step 1 : PG 승인 이벤트 생성

PG 승인 테이블 예시

```

payment_approval
-----
approval_id
merchant_id
card_no
amount
approval_time
status
pg_code
  
```

POC에서는 초당 200~500 TPS 정도 생성

테스트 데이터 생성

```
INSERT INTO payment_approval
VALUES (
    uuid(),
    rand_merchant(),
    rand_card(),
    rand_amount(),
    now(),
    'APPROVED',
    'PG01'
);
```

또는

```
kafka producer simulator
```

4 Step 2 : Kafka Ingest

Kafka Topic 설계

```
topic : pg_approval
partition : 6
replication : 3
```

메시지 구조

```
{
  "approval_id": "A12345",
  "merchant_id": "M1023",
  "amount": 15000,
  "approval_time": "2026-03-06 14:21:22",
  "status": "APPROVED"
}
```

Kafka 목표 TPS

```
1000 ~ 3000 TPS
```

5 Step 3 : Flink Streaming 처리

Flink에서 2가지 작업

1 Raw 데이터 Iceberg 적재

2 실시간 집계 생성

5-1 Raw Table (Iceberg)

```
CREATETABLE pg_approval_raw (  
  approval_id STRING,  
  merchant_id STRING,  
  amount BIGINT,  
  approval_timeTIMESTAMP,  
  status STRING  
)  
PARTITIONEDBY (days(approval_time))  
WITH (  
  'format-version'='2'  
);
```

Flink job

Kafka → Iceberg append

5-2 실시간 집계 테이블

```
CREATETABLE pg_approval_agg (  
  window_startTIMESTAMP,  
  merchant_id STRING,  
  approval_count BIGINT,  
  total_amount BIGINT  
)
```

Flink SQL

```
INSERTINTO pg_approval_agg  
SELECT  
  window_start,  
  merchant_id,  
  COUNT(*)AS approval_count,  
  SUM(amount)AS total_amount
```

```
FROMTABLE(  
  TUMBLE(TABLE pg_approval_raw,  
    DESCRIPTOR(approval_time),  
    INTERVAL '1' MINUTE))  
GROUPBY window_start, merchant_id;
```

이렇게 하면

1분 단위 승인 집계가 계속 업데이트

6 Step 4 : Iceberg 테이블 설계

Iceberg 설계가 성능 핵심입니다.

Raw Table

```
partition  
day(approval_time)
```

정산 테이블

```
month(approval_time)  
merchant_id bucket
```

예

```
PARTITIONEDBY (  
  months(approval_time),  
  bucket(16, merchant_id)  
)
```

이렇게 하면

```
정산 쿼리  
WHERE month='2026-03'
```

→ Partition pruning

7 Step 5 : Trino Query Engine

Trino는 BI Query 용

예시

실시간 승인 모니터링

```
SELECT
  merchant_id,
  SUM(approval_count),
  SUM(total_amount)
FROM pg_approval_agg
WHERE window_start> now()-interval'10'minute
GROUPBY merchant_id;
```

조회 속도

< 1 sec

승인 Top Merchant

```
SELECT
  merchant_id,
  SUM(amount) total_amount
FROM pg_approval_raw
WHERE approval_time>= date_trunc('day', now())
GROUPBY merchant_id
ORDERBY total_amountDESC
LIMIT10;
```

8 Step 6 : Spark 정산 배치

기존 RDBMS 정산은

대량 join + aggregation

Spark에서 수행

예

```
INSERT INTO settlement_table
SELECT
  merchant_id,
  date_trunc('day', approval_time) AS settle_date,
  SUM(amount) AS total_amount,
  COUNT(*) AS txn_count
FROM pg_approval_raw
GROUP BY merchant_id, settle_date;
```

Spark 장점

대량 parallel 처리

9 POC에서 반드시 보여줘야 하는 것

1 실시간성

승인 발생 → 3초 후 Dashboard

2 대용량 조회

예

10억건 조회

```
SELECT count(*) FROM pg_approval_raw;
```

Trino

2~5 sec

RDBMS

수십초 ~ 분

3 정산 배치 시간

예

기존

RDBMS batch

30분

Spark

3분

10 서버 스펙에서 튜닝 포인트

현재 서버

8core

12GB RAM

주의점

Flink

taskmanager.memory = 6GB

slot = 4

Spark

executor memory = 4GB

executor core = 2

Trino

```
query.max-memory=4GB
```

1 Iceberg 운영 필수 작업

POC라도 반드시 해야함

small file compaction

Spark job

```
rewrite data files
```

metadata cleanup

```
expire snapshots
```

1 POC 데모 시나리오

발표할 때 이렇게 보여주면 좋습니다.

Step1

승인 발생

```
Kafka TPS = 2000
```

Step2

Flink streaming ingest

```
Kafka → Iceberg
```

Step3

Dashboard

실시간 승인 그래프

Step4

대용량 조회

```
SELECT SUM(amount)
FROM 1year data
```

Step5

정산 batch

Spark job
3년 데이터
3분 처리

1 DBA 관점에서 이 아키텍처의 핵심

이 구조의 핵심은 3가지입니다.

Kafka → Streaming Ingest
Iceberg → Lakehouse Table
Trino → MPP Query

즉

Streaming + Lakehouse + MPP

 DBA 관점에서 말하면

이 아키텍처는 사실상 "**RDBMS OLTP + DW + Streaming**"을 통합한 구조입니다.

좋습니다. 지금 구성은 꽤 제대로 된 Lakehouse 스택이라서 POC를 성공시키려면 두 가지를 반드시 이해해야 합니다.

1 현재 아키텍처에서 90% 조직이 겪는 병목 3개

2 MinIO + Iceberg + Trino + Flink에서 반드시 해야 하는 튜닝 10개

이건 실제로 대기업 데이터레이크에서도 거의 동일하게 발생하는 문제입니다.

1 지금 아키텍처에서 가장 큰 병목 3개

현재 구조

```
Kafka
|
Flink
|
Iceberg
|
MinIO
|
Trino
```

여기서 병목은 99% 아래 3개입니다.

병목 **1** Small File Problem (Iceberg 치명적)

Flink streaming ingest는 파일을 계속 생성합니다.

예

```
1초마다 commit
```

그러면

```
1분 = 60 files
1시간 = 3600 files
하루 = 86400 files
```

이 상태에서 Trino가 쿼리하면

Trino → 86000 files scan

결과

query latency 폭발

POC에서 흔한 현상

10억 row인데
파일 200000개
→ query 30초

해결

반드시 **compaction job**

Spark

rewrite_data_files

예

```
CALL iceberg.system.rewrite_data_files(  
  table=>'pg.pg_approval_raw'  
);
```

권장

target file size

256MB

병목 MinIO Object Storage latency

RDBMS는

local disk

Lakehouse는

S3 API

MinIO 요청은

HTTP call

그래서

파일 1개 read = 10~30ms

파일 10000개면

$10000 \times 20\text{ms}$
= 200 sec latency

그래서 파일 개수가 곧 성능입니다.

해결

파일 사이즈

128MB ~ 512MB

병목 Iceberg Metadata Explosion

Iceberg는 snapshot 기반입니다.

Streaming ingest 하면

commit 계속 발생

예

1분마다 commit

하루

1440 snapshots

1달

43200 snapshots

그러면

metadata.json
manifest
manifest list

파일이 폭증합니다.

결과

query planning 느려짐

해결

metadata cleanup

```
CALL iceberg.system.expire_snapshots(  
  table=>'pg.pg_approval_raw',  
  older_than=>TIMESTAMP'2026-03-01'  
);
```

2 반드시 해야 하는 튜닝 10개

지금 서버 스펙

```
8core  
12GB RAM
```

이라서 더 중요합니다.

튜닝 1 Iceberg Partition 전략

절대 이렇게 하면 안됩니다.

```
partition by approval_time
```

좋은 설계

```
days(approval_time)
```

또는

```
months(approval_time)
```

예

```
PARTITIONEDBY (days(approval_time))
```

튜닝 2 bucket partition

정산 쿼리는 merchant 기준이 많습니다.

그래서

```
bucket(merchant_id)
```

예

```
PARTITIONEDBY (  
    months(approval_time),  
    bucket(16, merchant_id)  
)
```

튜닝 **3** Iceberg File Size

가장 중요

target file size

권장

256MB

설정

write.target-file-size-bytes

튜닝 **4** Flink checkpoint interval

default

10 sec

권장

60 sec

이유

Iceberg commit 줄이기

튜닝 5 Flink parallelism

지금 서버

8 core

권장

parallelism = 4

예

taskmanager slots = 4

튜닝 6 Kafka partition

Kafka 병목 피하려면

partition >= consumer parallelism

예

Kafka partition = 6

튜닝 7 Trino Worker Memory

지금 12GB 서버면

Coordinator

4GB

Worker

6GB

설정

```
query.max-memory  
query.max-memory-per-node
```

튜닝 **8** Trino Dynamic Filtering

Iceberg 쿼리에서 매우 중요

설정

```
enable-dynamic-filtering=true
```

튜닝 **9** Iceberg Metadata Cache

Trino

```
iceberg.metadata-cache.enabled=true
```

이거 안 하면

```
metadata scan 반복
```

튜닝 **10** Compaction Scheduler

Spark job

1시간마다 compaction

예

rewrite_data_files

3 PG 서비스에서 반드시 보여줘야 하는 POC 포인트

POC는 성능 차이를 보여주는 것입니다.

시나리오 1 : 실시간 승인 모니터링

Kafka → Flink → Iceberg → Trino

Latency

< 3 sec

시나리오 2 : 대용량 집계

예

1년 승인 데이터
10억 row

```
SELECT SUM(amount)
FROM pg_approval_raw
WHERE approval_time >= date '2025-01-01'
```

Trino

2~5 sec

RDBMS

수십초

시나리오 3 : 정산 배치

기존

RDBMS batch
40min

Spark

3min

4 지금 아키텍처에서 가장 위험한 것 하나

이건 경험적으로 거의 모든 POC가 실패하는 이유입니다.

Trino worker 2개

MPP 성능이 안 나옵니다.

권장

최소 worker 3~4

5 진짜 중요한 것 (DBA 관점)

이 아키텍처의 핵심은

Streaming layer → Flink
Storage layer → Iceberg
Query layer → Trino
Object storage → MinIO

즉

Streaming + Lakehouse + MPP

입니다.

좋습니다. PG 데이터 플랫폼 POC에서 임원/아키텍트들이 바로 “이거 기존 DB보다 훨씬 좋네”라고 느끼는 데모 시나리오는 보통 아래 3가지입니다.

핵심은 RDBMS가 약한 영역을 일부러 보여주는 것입니다.

1 실시간 이상 거래 감지 (Fraud / Spike Monitoring)

PG에서 가장 설득력 있는 실시간 시나리오입니다.

기존 RDBMS 구조

APP → DB INSERT
↓
배치 분석

문제

이상 거래 감지가 늦음

Lakehouse 시나리오

PG 승인 발생
|
Kafka
|

```
Flink (stream processing)
|
Iceberg
|
Trino Dashboard
```

Step 1 : 승인 이벤트 발생

Kafka topic

```
pg_approval
```

메시지

```
{
  "merchant_id": "M1001",
  "amount": 15000,
  "card_no": "1234-xxxx",
  "approval_time": "2026-03-06 14:21:22"
}
```

Step 2 : Flink 실시간 집계

예

1분간 승인 건수

```
SELECT
  merchant_id,
  window_start,
  COUNT(*) AS approval_cnt
FROM TABLE(
  TUMBLE(
    TABLE pg_approval,
    DESCRIPTOR(approval_time),
    INTERVAL '1' MINUTE))
GROUP BY merchant_id, window_start;
```

Step 3 : 이상 탐지

예

평소 1분 승인 50건

갑자기

1분 승인 500건

Flink rule

HAVINGCOUNT(*)>200

결과

Fraud alert

데모 포인트

대시보드

승인 TPS 그래프

승인 발생 후

3초 내 그래프 반영

RDBMS

거의 불가능

2 대용량 승인 데이터 초고속 조회

PG 회사에서 항상 있는 요구

1~3년 승인 데이터 분석

RDBMS 문제

데이터 수집역전

쿼리

```
SELECT SUM(amount)
FROM payment_approval
WHERE approval_time>='2024-01-01';
```

결과

수십초~분

Lakehouse 데모

Iceberg table

10억 ~ 50억 row

Trino query

```
SELECT
  merchant_id,
  SUM(amount)
FROM pg_approval_raw
WHERE approval_time>=DATE'2025-01-01'
GROUPBY merchant_id
ORDERBY2DESC
LIMIT10;
```

결과

3~5 sec

데모 포인트

화면에서

기간 선택

1일
1개월
1년
3년

조회 시간 거의 동일.

임원들이 여기서 놀랍니다.

3 정산 배치 초고속 처리

PG에서 정산 배치는 핵심입니다.

기존 구조

RDBMS

정산 쿼리

```
INSERT INTO settlement
SELECT
  merchant_id,
  trunc(approval_time),
  SUM(amount),
  COUNT(*)
FROM payment_approval
GROUP BY merchant_id, trunc(approval_time);
```

문제

데이터 수억건

배치 시간

30~60분

Lakehouse 방식

Spark

```
INSERT INTO settlement_table
SELECT
  merchant_id,
  date_trunc('day', approval_time) AS settle_date,
  SUM(amount),
  COUNT(*)
FROM pg_approval_raw
GROUP BY merchant_id, settle_date;
```

Spark parallel 처리

cluster 전체 사용

결과

3~5분

데모 포인트

발표할 때 이렇게 말하면 좋습니다.

기존 정산 배치

45분

Lakehouse

4분

4 추가로 하나 더

이건 **DBA들이 잘 안 떠올리는 데모**인데 매우 강력합니다.

승인 데이터 + 로그 + 외부 데이터 결합

예

승인 데이터
+
가맹점 정보
+
로그 데이터

RDBMS에서는

다른 시스템에 있음

Lakehouse

모두 S3/Iceberg

Trino

```
SELECT
  merchant_id,
  SUM(amount),
  COUNT(*)
FROM approval
JOIN merchant
ON approval.merchant_id= merchant.id
WHERE merchant.region='SEOUL'
GROUPBY merchant_id;
```

데모 메시지

데이터 사일로 제거

5 POC 데모 순서 (강력 추천)

발표 순서

1 실시간 승인 그래프

Kafka → Flink → Dashboard

실시간 반영

2 3년 데이터 조회

Trino query
3 sec

3 정산 batch 실행

Spark job

5분 완료

6 DBA 관점에서 가장 중요한 설명

임원에게 이렇게 설명하면 됩니다.

기존

OLTP DB
+
DW
+
Streaming

모두 분리.

Lakehouse

Streaming + DW + Data Lake

통합.

7 내가 하나만 더 말하면 (진짜 중요)

지금 구성에서 **POC 성공을 좌우하는 것 하나**가 있습니다.

Iceberg table 설계

이거 잘못하면

성능 100배 차이

납니다.

원하면 내가 지금 네 구성 기준으로

MinIO
Iceberg
Kafka
Flink
Trino
Spark

PG 데이터레이크 "추천 테이블 구조 + 파티션 전략"을 실제 SQL 수준으로 설계해 줄게.

이거 잘 짜면 **POC 성능이 최소 10배 차이** 납니다.